

Lehrziele:

- Methoden
- UnitTest
- 2-dimensionale Arrays

Aufgabenstellung

Das **Traveler-Salesman-Problem** ist ein Optimierung-Problem. Dabei soll für einen Handelsvertreter der kürzeste Weg gesucht werden, bei dem er alle seine Ziele zumindest einmal besucht.

Siehe : https://de.wikipedia.org/wiki/Problem_des_Handlungsreisenden

Das gesuchte Programm soll mit dem Brute-Force Verfahren den kürzesten Weg zu allen Zielen finden. Implementieren Sie die Berechnung mit der folgenden Methode:

```
int[] FindMinTravelSequence(double[,] locations, out double minDistance)
```

- **locations:** Der Parameter enthält alle Ziele, die der Vertreter besuchen muss. Die 2. Dimension des Array enthält die Koordinaten (x,y), daher ist die Größe 2. Beispiel: `new double[,] = { {1,1}, {2,1}, {2,2}, {1,2} }` besucht 4 Punkte.
- **minDistance:** Der Ausgabe Parameter teilt dem Aufrufer mit, wie weit der Vertreter fahren muss.
- **Rückgabewert:** Das `int[]` definiert die Ziele (Reihenfolge), die für den kürzesten Weg angefahren werden müssen. Die Werte (=Ziele) entsprechen dem Index der 1 Dimension des Parameters `locations`)

Für eine korrekte Umsetzung der Methode müssen die im Projekttemplate enthaltenen Unittests „grün“ werden. Änderungen in den Unittests sind nicht erlaubt.

Die Unittests enthalten auch Überprüfungen für Hilfsmethoden. Implementieren Sie diese Methoden (Sie könnten Ihnen bei der Umsetzung helfen).

```
double[,] CreateDistanceTable(double[,] locations)
```

- **locations:** Siehe `FindMinTravelSequence`.
- **Rückgabewert:** Eine Tabelle, die die Distanzen zwischen den einzelnen Zielen enthält. Beispiel: Das Element `[0,1]` enthält die Entfernung zwischen Ziel 0 und 1 (muss gleich sein wie `[1,0]`). Hinweis: `[x,x]` ist immer 0.0

```
double CalculateDistance(double[,] distanceTable, int[] travel)
```

- **distanceTable:** z.B. der Rückgabewert aus `CreateDistanceTable`.
- **travel:** Das Array enthält die Ziele, die nacheinander angefahren werden müssen. Hinweis: Der Vertreter muss immer zum Ausgangspunkt zurück!

```
int[][] CreatePermutations(int value)
```

- Diese Methode ist im Template bereits enthalten und muss nicht implementiert werden .

- Sie liefert alle Permutationen – also alle möglichen Varianten – wie der Vertreter die Ziele anfahren kann.
Beispiel: 4 liefert (es beginnt immer bei 0, es ist egal, wo der Vertreter seine Reise beginnt)
`{0, 1, 2, 3}`
`{0, 1, 3, 2}`
`{0, 2, 1, 3}`
`{0, 2, 3, 1}`
`{0, 3, 1, 2}`
`{0, 3, 2, 1}`
- Die Anzahl der Permutationen explodiert!
Ruft man die Methode mit 9 auf, gibt es bereits 40320 Permutationen - $\text{Fact}(9-1)$

Hauptprogramm

Schreiben Sie ein Hauptprogramm mit folgendem Ablauf

- Bei einer fehlerhaften Eingabe wird diese wiederholt.
- Der Anwender gibt als erstes die Anzahl der Ziele ein.
- Anschließend werden vom Anwender alle X und Y Werte eingegeben.
- Das Programm berechnet den optimalen Weg und gibt diesen detailliert aus.

BildschirmAusgabe

```
Traveler Salesman
*****
Please enter the count of locations [1..20]: 4
1. X-Position: 1
1. Y-Position: 1
2. X-Position: 10
2. Y-Position: 2
3. X-Position: 2
3. Y-Position: 1
4. X-Position: 11
4. Y-Position: 2
Shortest way to all destinations:
1: from 1 to 2 => 9,055385138137417
2: from 2 to 4 => 1
3: from 4 to 3 => 9,055385138137417
4: from 3 to 1 => 1
Total: 20,110770276274835
```